

2024 DL Lab4

Model Compression: Pruning & Quantization

The screenshot of training result from task 1:

Best Accuracy : 83.75

```
09 Nov 2024 13:18:14
Epoch: 28 |train loss: 0.0830 |train accuracy: 98.25 |validation loss: 0.5553 |validation accuracy: 83.75 |learning rate: 1.0e-03 |train time: 3.42 |test time: 1.69
Saving..
Training: 100%|██████████| 33/33 [00:03<00:00, 9.82it/s]
Testing: 100%|██████████| 5/5 [00:01<00:00, 2.94it/s]
09 Nov 2024 13:18:14
Epoch: 29 |train loss: 0.0669 |train accuracy: 98.93 |validation loss: 0.6269 |validation accuracy: 81.25 |learning rate: 1.0e-03 |train time: 3.36 |test time: 1.70
Best accuracy: 83.75
```

The screenshot of training result from task 2:

After fine-tune
Validation loss: 0.5172 Validation accuracy: 87.50

Explain Pruning briefly (What/Why/How)

What is Pruning?

Pruning 是一種模型壓縮技術，其主要目的是通過移除神經網路中較不重要的 weights 或是 channels 來減少模型的大小和計算成本，同時盡可能保持模型的性

Why Need Pruning?

1. 模型效率提升

- 減少模型大小：降低存儲空間需求
- 減少計算量：降低運算時的計算資源消耗
- 降低能耗：對於移動設備尤其重要

2. 實際應用需求

- 邊緣設備運算能力有限
- 即時性要求高的應用場景
- 存儲空間受限的環境

3. Model Generalization Ability

- 較小的模型可能具有更好的 generation ability
- 減少 overfittig 的風險

How to Implement Pruning?

1. 主要策略

- Structured Pruning :
 - 移除整個 channel 或 neuron
 - 不需要特殊的硬體支持
 - 實現較為簡單，但靈活性較低
- Unstructured Pruning :
 - 移除單個 weightedparameter
 - 可以達到更高的 sparsity
 - 需要特殊的硬體支持才能實現加速

2. 實現流程

1. 訓練原始模型 :
 - 先完整訓練得到性能良好的基礎模型
2. 評估重要性 :
 - 基於特定標準(如權重大小、BN scaling factor 等)評估 parameter 重要性
 - 決定 pruning threshold
3. 執行 pruning :
 - 移除低於 pruning threshold 的 parameter/channel
 - 重建網路結構
4. Fine-tuning :
 - 對 pruning 後的模型進行再訓練
 - 恢復模型性能

3. 具體實作方法

在本實驗中，我們採用基於 Batch Normalization (BN) 的 channel pruning 方法：

1. 重要性評估 :
 - 使用 BN 層的 scaling factor (γ) 作為 channel 重要性指標
 - γ 值越小，表示該 channel 對輸出的貢獻越小
2. pruning 過程 :

```
# you can choose your pruning rate
pruning_rate = 0.1
```

1.設定 pruning rate

```
# extract the weight of BatchNorm1d layer and sort them
```

```
total_par = sum([param.nelement() for param in net.parameters()])
print("Number of parameter: %.2fM" % (total_par/1e6))
```

```
total = 0
```

```
for m in net.modules():
    if isinstance(m, nn.BatchNorm1d):
        total += m.weight.data.shape[0]
```

```
bn = torch.zeros(total)
index = 0
for m in net.modules():
    if isinstance(m, nn.BatchNorm1d):
        size = m.weight.data.shape[0]
        # print(size)
        bn[index:(index+size)] = m.weight.data.abs().clone()
        index += size
```

2.根據 BN 權重大小排序

```
y, i = torch.sort(bn)
thre_index = int(total * pruning_rate)
thre = v[thre_index]
```

3.確定 pruning threshold

```
# record the renaming weight
```

```
pruned = 0
```

```
cfg = []
```

```
cfg_mask = []
```

```
for k, m in enumerate(net.modules()):
    if isinstance(m, nn.BatchNorm1d):
        weight_copy = m.weight.data.abs().clone().cpu()
```

```
# if weight larger than threshold
```

```
mask = weight_copy.gt(thre).float().cuda()
```

4.移除小於 threshold 的 channel

```
# pruning number
```

```
pruned = pruned + mask.shape[0] - torch.sum(mask)
```

```
m.weight.data.mul_(mask)
```

```
m.bias.data.mul_(mask)
```

```
cfg.append(int(torch.sum(mask)))
```

```
cfg_mask.append(mask.clone())
```

```
print('layer index: {:d} \t total channel: {:d} \t remaining channel: {:d}'.format(k, mask.shape[0], int(torch.sum(mask))))
```

```
elif isinstance(m, nn.AvgPool1d):
```

```
    cfg.append('P')
```

```
# print('cfg', cfg)
```

```
pruned_ratio = pruned/total
```

```
print('Pre-processing Successful!')
```

3. 迭代式 pruning :

- 進行多次 pruning-微調循環
- 每次 pruning 後進行模型微調
- 逐步提高模型壓縮率

通過這種方式，我們成功將模型大小降低的同時，依然保持了較好的性能表現。在本實驗中，經過兩次 pruning 後模型 parameter 量從 0.27M 降低到 0.17M，而準確率仍然維持在 87.5% 左右。

The screenshot of training result from task 3:

Best Accuracy : 74.38

```
09 Nov 2024 17:23:48
Epoch: 197 |train loss: 0.4196 |train accuracy: 88.51 |validation loss: 0.9926 |validation accuracy: 70.62
09 Nov 2024 17:23:53
Epoch: 198 |train loss: 0.4296 |train accuracy: 88.02 |validation loss: 1.0616 |validation accuracy: 71.88
09 Nov 2024 17:23:57
Epoch: 199 |train loss: 0.5542 |train accuracy: 87.93 |validation loss: 1.0590 |validation accuracy: 72.50
Best accuracy: 74.38
```

Explain Quantization briefly (What/Why/How)

What is Quantization?

Quantization 是一種模型壓縮技術,主要是將 Neural Network 中較高精度的浮點數(例如 32 位元浮點數)轉換為較低精度的整數或浮點數(例如 8 位元整數)。這種轉換過程包括:

1. weights 量化: 將模型中的 weights 從浮點數轉換為整數
2. activation 量化: 將運算過程中的 activations 也進行量化處理
3. 運算量化: 使用低精度運算替代高精度運算

Why Need Quantization?

量化主要具有以下優點:

1. 降低模型大小
 - 減少模型存儲空間需求
 - 有助於部署到記憶體受限的裝置
 - typically 可以將模型大小縮小到原本的 1/4

2. 提升運算速度

- 整數運算比浮點運算更快
- 減少記憶體存取頻寬需求
- 可降低功耗,特別適合移動裝置

3. 硬體友好

- 許多硬體加速器優化了整數運算
- 更適合邊緣運算裝置
- 有助於實現低延遲

How to Implement Quantization?

實現量化主要有三種方法:

1. Post-Training Quantization

- 直接將訓練好的模型進行量化
- 最簡單但精度損失較大
- 範例

```
#EXAMPLE
model_fp32 = load_model()
model_int8 = quantize(model_fp32, bits=8)
```

2. Quantization-Aware Training

- 在訓練過程中模擬量化效果
- 可以減少量化帶來的精度損失
- 範例:

```
#EXAMPLE
class QuantizedConv2d(nn.Module):
    def __init__(self, bits=8):
        self.bits = bits

    def forward(self, x):
        # 在前向傳播中模擬量化
        x = quantize(x, self.bits)
        return x
```

3. Dynamic Quantization

- 僅在推論時進行量化
- weights 提前量化,activation 即時量化
- 範例:

```
#EXAMPLE
model_int8 = torch.quantization.quantize_dynamic(
    model_fp32,
    {nn.Linear},
    dtype=torch.qint8
)
```

在本實驗中,我們採用 Quantization-Aware Training:

量化是通過 SincNet_Quat 模型中的量化層來實現的:

```
SincNet_Quat(
  (sincconv): _Layer( 1.權重量化：使用 7-bit quantization 對 SincConv 層的權重進行量化
    (conv): QuatSincConv1d(
      (w_quan): Quantize(num_of_bits=7)
    )
    (logabs): LogAbs()
    (relu): ReLU()
    (quan): Quantize(num_of_bits=8)
    (bn): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (pool): AvgPool1d(kernel_size=(2,), stride=(2,), padding=(0,))
  )
  (features): ModuleList(
    (0): _Layer( 2. activation 量化：使用 4-bit quantization 對中間層的輸出進行量化
      (conv0): QuaternaryConv1d(32, 32, kernel_size=(25,), stride=(2,), bias=False)
      (conv1): QuaternaryConv1d(32, 32, kernel_size=(25,), stride=(2,), bias=False)
      (bn): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (quan): Quantize(num_of_bits=4)
      (relu): ReLU()
      (pool): AvgPool1d(kernel_size=(2,), stride=(2,), padding=(0,))
    )
    (1): _Layer(
      (conv0): QuaternaryConv1d(32, 32, kernel_size=(9,), stride=(1,), bias=False)
      (conv1): QuaternaryConv1d(32, 64, kernel_size=(9,), stride=(1,), bias=False)
      (bn): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (quan): Quantize(num_of_bits=4)
      (relu): ReLU()
      (pool): AvgPool1d(kernel_size=(2,), stride=(2,), padding=(0,))
    )
    (2-4): 3 x _Layer(
      (conv0): QuaternaryConv1d(64, 64, kernel_size=(9,), stride=(1,), bias=False)
      (conv1): QuaternaryConv1d(64, 64, kernel_size=(9,), stride=(1,), bias=False)
      (bn): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (quan): Quantize(num_of_bits=4)
      (relu): ReLU()
      (pool): AvgPool1d(kernel_size=(2,), stride=(2,), padding=(0,))
    )
  )
  (gap): AdaptiveAvgPool1d(output_size=1)
  (quan_gap): Quantize(num_of_bits=8)
  (fc): Linear(in_features=64, out_features=10, bias=True)
)
```

3. 實驗效果

- 量化前模型大小: 0.27M parameter
- 量化後精確度: 驗證集準確率達到 74%
- 成功在保持可接受精度的前提下壓縮了模型

Compare the difference from above methods of compression

一、實驗概述

本實驗實作並比較了三種主流模型壓縮方法：

1. SincNet (Baseline)
2. Network Pruning
3. Model Quantization

這三種方法分別從不同角度對模型進行優化，以下進行詳細比較分析。

二、方法比較

2.1 原理比較

1. 原始 SincNet

- 使用完整的網路架構
- 使用 32 位元浮點數進行計算
- 保留所有 parameter 和 channel
- 模型大小和計算量最大

2. Pruning

- 基於重要性評估移除不重要的 channel/parameter
- 使用 Batch Normalization 層的縮放因子(γ)評估 channel 重要性
- 採用迭代式 pruning + 微調策略
- 保持網路基本結構，但降低複雜度

3. Quantization

- 將高精度浮點數轉換為低精度整數
- 不改變網路結構
- 主要透過降低數值精度來壓縮
- 權重和激活值都進行量化處理

2.2 效果比較

方法	模型大小	Validation Acc.	壓縮比例
SincNet (Baseline)	0.27M	83.75%	1x
Pruning(第一次)	0.21M	80.00%	1.29x
Pruning(第二次)	0.17M	87.50%	1.59x
Quantization	約 0.07M	74.38%	約 4x

*注：量化後的模型大小為估算值，基於 8-bit 量化計算

2.3 優缺點分析

1. Network Pruning

優點：

- 直接減少 parameter 數量和計算量
- 準確率損失較小
- 可以根據需求調整 pruning 強度
- 通過反覆微調可以恢復性能

缺點：

- 需要較長的微調時間
- pruning 過程較為複雜
- 需要專門的 sparse computing 支持

2. Model Quantization

優點：

- 壓縮比例大
- 實現相對簡單
- 推理速度快
- 硬體友好

缺點：

- 準確率損失較大
- 訓練過程需要特別處理
- 對硬體支持有要求

三、實驗結果分析

1. 壓縮效果

- 量化提供最高的壓縮比(約 4 倍)
- pruning 提供較溫和的壓縮(1.29x-1.59x)
- 兩種方法都能顯著減少模型大小

2. 準確率影響

- pruning 後模型準確率維持甚至提升
- 量化導致準確率下降
- pruning 在準確率保持方面表現更好

3. 實用性考量

- pruning 適合對準確率要求高的場景
- 量化適合對存儲空間和推理速度要求高的場景
- 兩種方法可以結合使用以達到更好的壓縮效果

四、結論與建議

1. 應用場景選擇

- 高精度需求：建議使用 pruning
- 資源受限場景：建議使用量化
- 可以考慮結合兩種方法，先 pruning 再量化

2. 性能權衡

- pruning 在保持性能方面表現更好
- 量化在資源節省方面效果更明顯
- 需根據具體應用需求選擇適當方法

本次實驗證明了不同壓縮方法各有優勢，在實際應用中應當根據具體需求選擇合適的壓縮策略，或採用混合方案以實現最佳效果。